

DopaPal

AI & NLP Component

Documentation & Backend Integration Guide

Prepared by

Mostafa Abdelhamed

Module location

`server/app/services/ai/`

1. Overview & Purpose of the AI & NLP Component

1.1 What this component is

The AI & NLP component is the part of dopaPal responsible for turning a user's messy, unstructured input a highlighted sentence, a voice dump, a quick typed note into a clean, structured task the rest of the app can schedule and reason about. It also decides which task to surface next, based on the PINCH engagement framework, rather than simple deadline sorting.

It lives entirely inside server/app/services/ai/ and has no direct dependency on the database, FastAPI, or any other backend service. It is a pure logic layer: text and numbers go in, validated structured data comes out. This separation is intentional: it lets the AI module be built, tested, and demoed independently of the rest of the backend team's work, and lets any other service (the FastAPI route, a future CLI tool, a test script) call it the same way.

1.2 Why it is built this way: deterministic-first

A core design decision is that task creation must never depend on an LLM being available. ADHD-focused productivity tools fail their users badly if a core feature breaks because a local model server is down. So the module is built in two layers:

- A deterministic layer (regex, dateparser, keyword matching) that runs every time and always produces a valid result, even with a vague or incomplete input.
- An optional LLM enrichment layer (**local Ollama model**) that can refine a title or estimate difficulty, but can only ever improve on the deterministic result, never block it, delay it past a timeout, or cause it to fail.

This means the AI component is safe to demo live even without Ollama running the deterministic pipeline alone produces correct results.

1.3 What it actually does, end to end

1. Ingests raw text (from a highlight, voice transcript, or manual entry) and extracts: a clean task title, a deadline, an effort estimate in hours, and an interest tag.
2. Splits the task into 2-hour sub-blocks and spreads them across the time remaining until the deadline, instead of clustering them right before the cutoff (the project's core anti-procrastination mechanic).
3. Scores every task using the PINCH framework (Passion, Interest, Novelty, Challenge, Hurry) so that the “next task” the user sees is the most engaging one, not just the most urgent one and shifts that scoring on low-energy days to favor easier, more interesting tasks.
4. Optionally calls a local Ollama model to refine the title or estimate difficulty, with automatic, silent fallback to the deterministic result if Ollama is unavailable or returns something unusable.

2. The Files I Built : What Each One Does

All files below live under *server/app/services/ai/* (the AI module itself) and *server/app/tests/services/ai/* (its test suite). Nothing here was placed inside any other team member's folders.

2.1 Module root : *server/app/services/ai/*

File	What it does
<code>__init__.py</code>	Marks the folder as a Python package and re-exports the main classes (ParsedTask, SubBlockPlan, IngestResult, AIService) so other code can import them with one clean line.
<code>schemas.py</code>	Defines every data shape (Pydantic models) the module uses or returns: ParsedTask, SubBlockPlan, IngestResult, PinchInput, StateContext. This is the contract layer the backend team's code should rely on these shapes rather than raw dictionaries.
<code>ingestion.py</code>	The IngestionPipeline class. Takes raw text + source type, runs it through the deterministic parsers, optionally enriches it with the LLM, and returns one validated ParsedTask. This is the “NLP Ingestion Engine” from the README's architecture diagram.
<code>chunking.py</code>	The ChunkingEngine class. Splits a task's estimated hours into 2-hour sub-blocks and spreads their scheduled dates across the window between now and the deadline, so work is never crammed at the last minute.
<code>pinch.py</code>	The PinchEngine class. Computes the “Selection Priority Score” for a task using four weighted signals Urgency, Novelty, Challenge, Interest with the weighting itself shifting depending on the user's daily energy state.
<code>service.py</code>	The AIService class is the single entry point everyone outside the AI module should use. Exposes two methods: ingest() (full text-to-task pipeline) and score_for_bubble() (ranks tasks for the “next task” bubble).

2.2 Deterministic parsers : *server/app/services/ai/parsers/*

File	What it does
<code>__init__.py</code>	Marks the parsers folder as a Python package.
<code>date_parser.py</code>	Extracts a deadline from free text using the dateparser library plus a custom “end of day” detector for phrases like “tonight”. Falls back to a default of +7 days if nothing parseable is found.
<code>effort_parser.py</code>	Extracts an effort estimate in hours.
<code>interest_tagger.py</code>	Matches the task text against a keyword taxonomy (architecture, cybersecurity, programming, writing, language_learning, ai_ml, design, research) to assign an interest tag. This taxonomy also powers the future Interest Vault feature.
<code>title_sanitizer.py</code>	Cleans the raw text into a proper task title strips filler phrases (“I need to...”) and trailing deadline/effort clauses, while preserving acronyms like “API” or “NLP” instead of mangling them with title-casing.
<code>challenge_estimator.py</code>	A keyword- and structure-based heuristic that estimates how difficult a task sounds (low / medium / high), used to make PINCH's “Challenge” signal a real, honest value instead of a hardcoded constant.

2.3 LLM extension point : server/app/services/ai/llm/

File	What it does
<code>__init__.py</code>	Marks the llm folder as a Python package and exports OllamaClient and OllamaUnavailableError.
<code>ollama_client.py</code>	A thin wrapper around a local Ollama server's /api/generate endpoint. Sends the raw task text and asks for an optional refined title and difficulty rating, strictly as JSON. Any failure (timeout, server down, bad output) raises OllamaUnavailableError, which calling code is required to catch and gracefully ignore.

2.4 Test suite : server/app/tests/services/ai/

Every file above has a matching pytest file. The suite uses dependency-injected “current time” values (a fixed_now fixture) instead of a time-freezing library, and mocks the Ollama HTTP layer directly so the tests never require a live Ollama server to pass.

File	What it does
<code>conftest.py</code>	Shared pytest fixtures are currently just fixed_now, a fixed Tuesday timestamp so date-parsing tests are 100% reproducible regardless of when the suite is run.
<code>test_date_parser.py</code> / <code>test_effort_parser.py</code> / <code>test_interest_tagger.py</code> / <code>test_title_sanitizer.py</code> / <code>test_challenge_estimator.py</code>	Unit tests for each individual deterministic parser, covering both the happy path and the “no signal found, use the default” fallback path.
<code>test_chunking.py</code>	Tests that sub-blocks are sized and spread correctly, including edge cases: a task too short for a full block, a remainder too small to stand alone, and an already-overdue deadline.
<code>test_pinch.py</code>	Tests the scoring weights at high vs. low energy, urgency behavior near vs. far from a deadline, and that an explicit challenge hint correctly overrides the keyword heuristic.
<code>test_ingestion.py</code>	End-to-end tests of the full pipeline, plus tests proving the LLM path is never called unless explicitly enabled, and that an LLM failure never breaks ingestion.
<code>test_ollama_client.py</code>	Tests the Ollama HTTP client against a mocked transport success, connection failure, malformed JSON, and unexpected-key stripping with zero dependency on a real Ollama process.
<code>test_service.py</code>	Tests the top-level AIService class: full ingest() output shape, that LLM is never touched when disabled, ranking behavior, and that the same input always produces the same output (determinism).

3. Backend Integration Guide

This section is written for whoever on the backend team is wiring the AI module into the live FastAPI app. Follow it in order.

3.1 What you need from this module

In almost every case, you only need to import one thing:

```
from app.services.ai.service import AIService, IngestOptions
from app.services.ai.schemas import SourceType, ParsedTask, IngestResult, PinchInput
```

You should not need to import anything from parsers/, llm/, ingestion.py, chunking.py, or pinch.py directly. AIService is the single, intended entry point.

3.2 Step 1: Install dependencies

Two new dependencies are required by this module: dateparser (deterministic date parsing) and httpx (the Ollama HTTP client). Add them to server/pyproject.toml:

```
dependencies = [
    # ...existing deps...
    "dateparser>=1.2.0",
    "httpx>=0.27.0",
]

[dependency-groups]
dev = [
    # ...existing dev deps...
    "pytest-mock>=3.14.0",
]
```

Then run, from the server/ directory:

```
uv sync
```

3.3 Step 2 : Run the test suite first

Before wiring anything into a live route, confirm the module works in isolation. From server/:

```
uv run pytest app/tests/services/ai -v
```

All tests should pass. If any fail most likely candidate is a date-parsing edge case in dateparser flag it to the AI engineer (Mo) before proceeding, rather than working around it in the integration layer.

3.4 Step 3 : Create the ingest endpoint

Add (or update) server/app/api/tasks.py with a route that calls AIService.ingest(). This endpoint corresponds to the project's documented contract: POST /api/v1/tasks/ingest.

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
```

```

from app.services.ai.service import AIService, IngestOptions
from app.services.ai.schemas import SourceType

router = APIRouter()
ai_service = AIService(IngestOptions(use_llm=False))

class TaskIngestRequest(BaseModel):
    source_text: str
    source_type: SourceType

@router.post("/api/v1/tasks/ingest", status_code=201)
def ingest_task(payload: TaskIngestRequest):
    try:
        result = ai_service.ingest(
            raw_text=payload.source_text,
            source_type=payload.source_type,
        )
    except ValueError as exc:
        raise HTTPException(status_code=422, detail=str(exc)) from exc

    # Persist result.title / result.deadline / result.estimated_hours /
    # result.interest_tag to the `tasks` table, then persist each entry
    # in result.sub_blocks to the `sub_blocks` table, attaching the new
    # task's database id as task_id on every sub-block.
    # Return the README's documented response shape, with the new
    # database-assigned `id` included.
    ...

```

Note the single try/except ValueError block this is the only case the AI module intentionally raises on (completely empty input). Every other messy or vague input still produces a valid, schedulable task rather than an error.

3.5 Step 4 : Wire the “next task” bubble endpoint

This corresponds to GET /api/v1/bubble/next. The state service computes a 0–100 state_score using the compute_state_score() function from the README; this score is then passed into the AI module's ranking method:

```

candidates: dict[int, PinchInput] = {
    task.id: PinchInput(
        deadline=task.deadline,
        created_at=task.created_at,
        interest_tag=task.interest_tag,
        user_interest_tags=current_user.interest_tags,
        estimated_hours=task.estimated_hours,
        raw_source_text=task.raw_source_text,
    )
    for task in candidate_tasks
}

ranked = ai_service.score_for_bubble(candidates, state_score=state_score)
primary_task_id, primary_score = ranked[0]

```

```
# Per the project's Design Rule #1: only surface up to 2 bonus blocks
# when state_score >= 50. This display decision belongs here, in the
# backend route – the AI module only ranks, it does not decide how
# many results to show.
bonus = ranked[1:3] if state_score >= 50 else []
```

3.6 Step 5 : (Optional, later) enable the Ollama LLM enrichment

The module ships with `use_llm=False` by default fully deterministic, no dependency on Ollama being installed. Once Ollama is confirmed running locally and the team wants the LLM refinement step active, flip one flag:

```
ai_service = AIService(IngestOptions(use_llm=True))
```

Optionally, point it at the project's .env values (`AI_USE_LLM`, `OLLAMA_BASE_URL`, `OLLAMA_MODEL`) instead of a hardcoded flag, once a central `app/core/config.py` settings object exists the AI module's `OllamaClient` currently uses sane hardcoded defaults (localhost:11434, model llama3.1) and does not read environment variables itself.

3.7 Step 6 : Sanity-check the integration

5. Start the backend (uv run dopapal or via Docker Compose).
6. POST the README's example payload to `/api/v1/tasks/ingest`:

```
{
  "source_text": "Need to complete the system specification document by next Friday
night, should take about 6 hours total.",
  "source_type": "highlight"
}
```

7. Confirmed the response includes a clean title (“Complete the system specification document”), a deadline next Friday at 23:59:59, an `estimated_hours` of 6.0, an `interest_tag` of “architecture”, and 3 sub-blocks spread across the window before that Friday.
8. Confirm the task and its sub-blocks were persisted correctly in PostgreSQL.
9. Call `/api/v1/bubble/next` and confirm a `primary_block` is returned, with `bonus_blocks` only appearing when the test user's `state_score` is 50 or above.

4. Complete Project File Structure

Full repository layout, with the AI & NLP component's files highlighted in context. Bracketed notes describe each top-level area's purpose; the AI module's own files are listed in full since that is the component this document covers.

```
dopapal/
├── client/                                [Electron desktop app - frontend team]
│   └── src/
│       ├── main/                         (system hooks, window managers)
│       └── renderer/                     (React dashboard & overlay UI)
├── server/                               [FastAPI backend - shared]
│   ├── app/
│   │   ├── api/                         [Route controllers - backend team]
│   │   │   └── tasks.py                 (calls AIService.ingest(), see §3.4)
│   │   ├── core/                       [Config, DB setup, JWT auth - backend team]
│   │   └── models/                     [SQLAlchemy schemas - backend team]
│   ├── services/
│   │   └── ai/                          *** THIS COMPONENT (AI & NLP) ***
│   │       ├── __init__.py             package exports
│   │       ├── schemas.py              Pydantic data contracts
│   │       ├── ingestion.py            IngestionPipeline (text → ParsedTask)
│   │       ├── chunking.py             ChunkingEngine (task → sub-blocks)
│   │       ├── pinch.py                PinchEngine (PINCH scoring)
│   │       ├── service.py              AIService (single entry point)
│   │       └── parsers/
│   │           ├── __init__.py
│   │           ├── date_parser.py
│   │           ├── effort_parser.py
│   │           ├── interest_tagger.py
│   │           ├── title_sanitizer.py
│   │           └── challenge_estimator.py
│   │       └── llm/
│   │           ├── __init__.py
│   │           └── ollama_client.py
│   └── tests/
│       └── services/
│           └── ai/                      *** AI MODULE TEST SUITE ***
│               ├── __init__.py
│               ├── conftest.py
│               ├── test_date_parser.py
│               ├── test_effort_parser.py
│               ├── test_interest_tagger.py
│               ├── test_title_sanitizer.py
│               ├── test_challenge_estimator.py
│               ├── test_chunking.py
│               ├── test_pinch.py
│               ├── test_ingestion.py
│               ├── test_ollama_client.py
│               └── test_service.py
```


└─ main.py	[App entry point – backend team]
└─ migrations/	[Alembic DB migrations – backend team]
└─ pyproject.toml	[Add dateparser, httpx, pytest-mock – \$3.2]
└─ uv.lock	
└─ Dockerfile	
└─ .env.example	[Add AI_USE_LLM / OLLAMA_* vars - \$3.6]
└─ docker-compose.yml	
└─ README.md	
└─ run.sh / run.bat	
└─ Anchor_Product_Vision_and_UX_Documentation.md	